

Algorithms



Lecture 2 Searching with Divide and Conquer

Prantik Paul [PNP]

Lecturer

Department of Computer Science and Engineering
BRAC University

Linear Search

- The most basic searching algorithm
- Given an item to search from a bucket of items, we search ONE BY ONE.
- If we find then bingo! Else we look till the end of bucket.
- Example: Given an array of items. We will try to find “xyz” from it using linear search.

0	1	2	3
cxc	dfd	dfd	dfd

Linear Search

Simulation (Iterative)

- As mentioned in the previous slide linear search is done one by one hence we start searching for “xyz” from index 0. If the items match we stop else we look for the next item in the array.

0	1	2	3
cxc	dfd	dfd	dfd
xyz	xyz	xyz	xyz

✗ ✗ ✗ ✗

Linear Search

0	1	2	3
cxc	dfd	xyz	dfd
xyz	xyz	xyz	
✖	✖	😊	

Linear Search

Pseudo code (Iterative)

```
boolean linearSearch(A[], item){  
    int i = 0;  
    while(i < A.length){  
        if (A[i] matches item){  
            return true;  
        }  
        i++;  
    }  
    return false;  
}
```

Starting from index 0 of the array A

Loop to traverse till the end of the array

Checking if the *i*th element of the array matches with *item*

If matches, then return true, not searching further.

If mismatch in the if condition, this line will run moving to the next index of the array.

The program will reach this line after while loop is complete. This means all indices are traversed and item could not be found. False is returned.

Linear Search

Time Complexity (Iterative)

```
boolean linearSearch(A[], item){  
  
    int i = 0;  
    while(i < A.length){  
        if (A[i] matches item){  
            return true;  
        }  
        i++;  
    }  
    return false;  
}
```



0	1	2	3
cxc	dfd	dfd	dfd
×	×	×	×

The worst case scenario occurs when the loop runs from index 0 to the end of the array. That is the item is found in the last index or not found at all. Recall the above scenario.

If there are n indices in an array, the worst case would be $O(n)$

Now for the best case to occur the item we are searching for appears at the beginning as a result the number of search is just 1. This is an once in a blue moon situation must be ignored.

Divide and Conquer Strategy

Divide

the problem (instance) into subproblems.

Conquer

the subproblems by solving these recursively (or iteratively).

Combine

the solutions to the subproblems.

Binary Search

- A divide and conquer algorithm
- Does not search the entire array like linear search
- Given an array of **sorted elements** and the item, k , to search for, we first check if the middle element matches k .
- If we find then bingo! Else we check if $k > \text{middle element}$ or $k < \text{middle element}$?
If the first condition is true we look into the right half of the array. If the second condition is true we look into the left half of the array.
- Note that the red marked conditions are true if and only if the array is sorted.

Binary Search

Pseudo code (Iterative)

```
boolean binarySearch1D(A[], l, r, item){  
    while (l <= r){  
        mid = (l+r)/2;  
        if (A[mid]==item){  
            return true;  
        }else{  
            if (item < A[mid]){  
                r = mid-1;  
            }else{  
                l = mid+1;  
            }  
        }  
    }  
    return false;  
}
```

l must always be less or equal to r for the loop to run

Find the mid index

Check if item matches with the mid element, return true if does else next step.

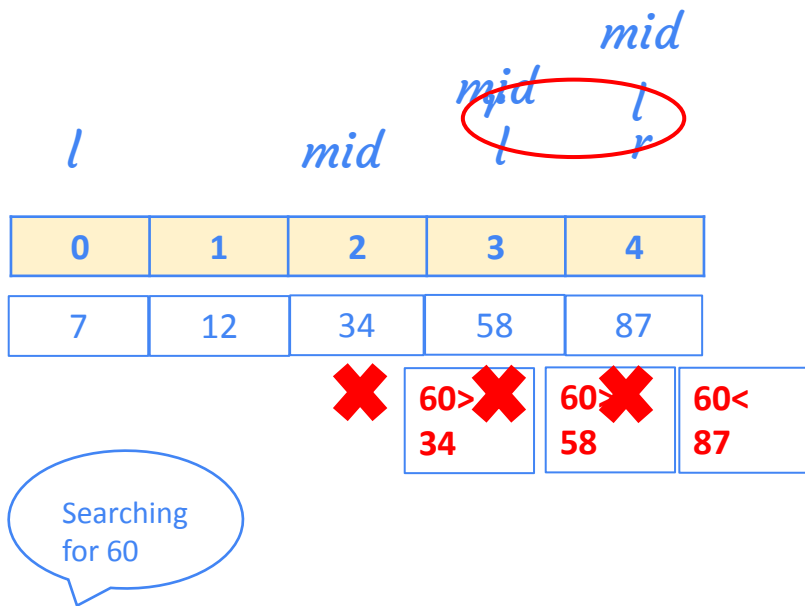
Check if the item is smaller, if yes shift the search range to left by moving r

If the above condition is dissatisfied, shift the search range to right by moving l

The program will reach this line after while loop is complete. False is returned.

Binary Search

Simulation



```
boolean binarySearch(A[], l, r, item){
    while (l <= r){
        mid = (l + r) / 2;
        if (A[mid] == item){
            return true;
        } else {
            if (item < A[mid]){
                r = mid - 1;
            } else {
                l = mid + 1;
            }
        }
    }
    return false;
}
```

Binary Search

Time Complexity (Iterative)

In order to compute the time complexity of divide and conquer problems we will focus on the problem size and **work done** at every step. Work done refers to finding the mid, matching, changing left or right. These are constant operations. Refer to the previous slide. You will see that at each step the problem size (range) was reducing by factor of 2.

Initially the domain of search was the entire array of length n . In the second step we only considered the right half. The domain got reduced to $n/2$. In the following steps this sub problem was getting smaller by factor of 2; $n/4, n/8, \dots$ until 1 or 0. We need to count how many steps it took for the problem of size n to become 0 or 1 then multiply it by work done at each step.

Problem Size	Step No.
n	0
$n/2$	1
$n/4$	2
$n/8$	3
....	
1	k

Binary Search

Time Complexity (Iterative) Contd.

Problem Size	Step No.	Work done at each step
n	0	1
n/2	1	1
n/4	2	1
n/8	3	1
....		1
1	k	1

It took k steps for the searching to end and work done at each step was constant. We need to find k in terms of n . If you notice the divisors of the problem size (marked red) are all powers of 2 and we can use the step numbers as exponents. Therefore each problem size can be written as,

$n/2^{(\text{step no.})}$

The last line can be written as, $1 = n/2^k$.

If we solve it, we will find that $k = \log_2 n$

Therefore the time complexity is $\log_2 n \times 1$, which eventually is $O(\log_2 n)$

Binary Search

→ Finding upper and lower bound

Input : arr[] = {1, 3, 5, 5, 5, 5, 67, 123, 125}, x = 5

Output : First Occurrence = 2

Last Occurrence = 5

Ternary Search

- A divide and conquer algorithm
- Very similar to binary search except that the array is broken down into 3 portions
- There are 2 “mids” and we check for the item, k inside both of them
- If we find then bingo! Else we check if $k < \text{first mid}$ or $k > \text{second mid}$ or $k > \text{first mid}$ and $k < \text{second mid}$? If the first condition is true we look into the rightmost portion of the array. If the second condition is true we look into the leftmost portion of the array else we look into the centre portion.
- Note that the red marked conditions are true if and only if the array is sorted.

Ternary Search

Pseudo code (Iterative)

```
boolean TernarySearch(A[], l, r, item){  
    while (l <= r){  
        mid1 = l + (r - l)/3;  
        mid2 = r - (r - l)/3  
        if (A[mid1]==item || A[mid2]==item){  
            return true;  
        }else{  
            if (item < A[mid1]){  
                r = mid1-1;  
            }elseif (item > A[mid2]){  
                l = mid2+1;  
            }else{  
                l = mid1+1; r = mid2-1;  
            }  
        }  
    }  
    return false;  
}
```

l must always be less or equal to r for the loop to run

Find the mid1 and mid2 index

Check if item matches, return true if does else next step.

Check if the item is smaller, if yes shift the search range to leftmost portion by moving r

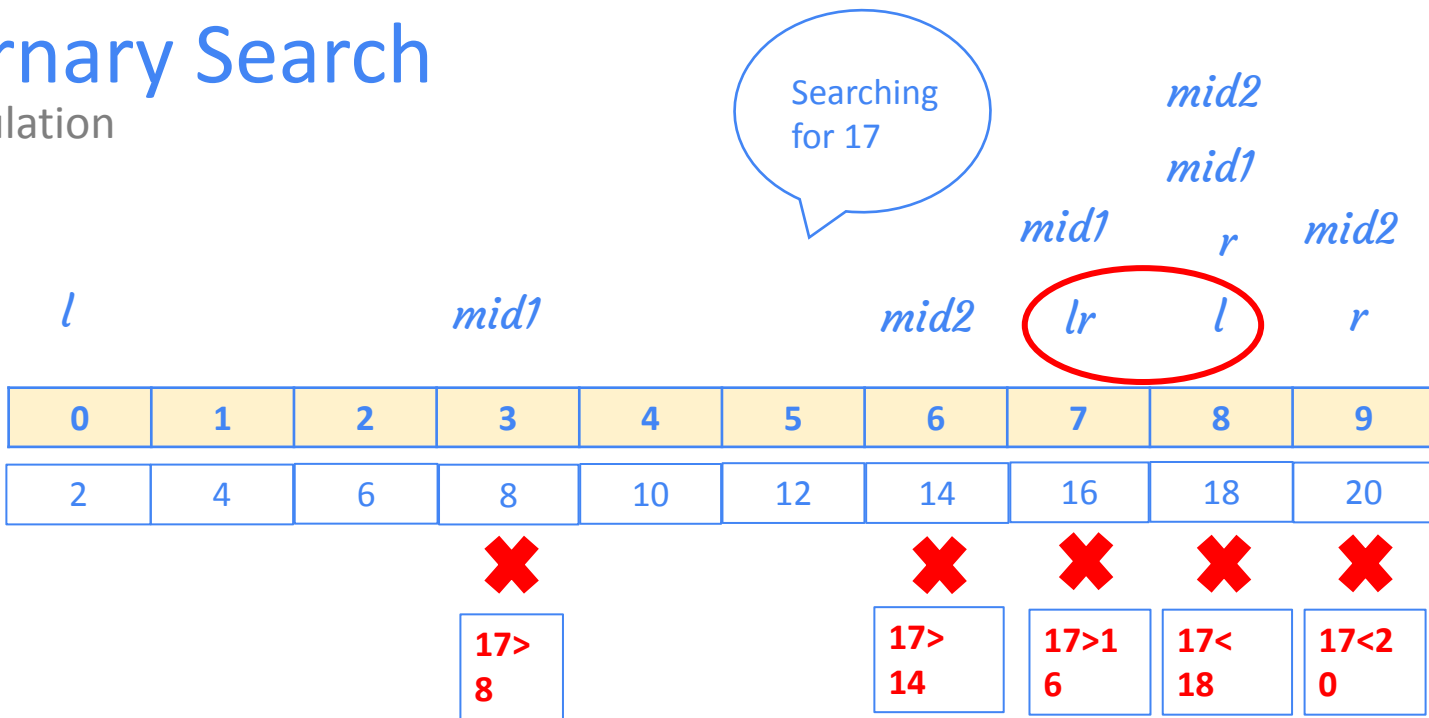
If the above condition is dissatisfied, shift the search range to rightmost portion by moving l

Else shift the search domain to the middle portion

The program will reach this line after while loop is complete. False is returned.

Ternary Search

Simulation



Ternary Search

Time Complexity (Iterative)

We have already learnt or at least got a good idea to find the time complexity of divide and conquer problems from the binary search slide. Refer to the algorithm and simulation of ternary search, you must get the concept that this and the binary search are almost the same saving the fact that at each step the problem size (range) is reducing by factor of 3.

Initially the domain of search was the entire array of length n . At each step the search space was getting smaller by factor of 3 until 1 or 0. Find the running is same as binary search.

Problem Size	Step No.
n	0
$n/3$	1
$n/9$	2
$n/27$	3
....	
1	k

Ternary Search

Time Complexity (Iterative) Contd.

Problem Size	Step No.	Work done at each step
n	0	1
$n/3$	1	1
$n/9$	2	1
$n/27$	3	1
....		1
1	k	1

It took k steps for the searching to end and work done at each step was constant. We need to find k in terms of n . If you notice the divisors of the problem size (marked red) are all powers of 3 and we can use the step numbers as exponents. Therefore each problem size can be written as,

$n/3^{(\text{step no.})}$

The last line can be written as, $1 = n/3^k$.

If we solve it, we will find that $k = \log_3 n$

Therefore the time complexity is $\log_3 n \times 1$, which eventually is $O(\log_3 n)$

Quaternary Search

- Divide in 4 parts?
- Pros?
- Cons?
- What about N-ary Search?